

Unix processzek, ütemezés

Az előadás anyaga

Dr. Vadász Dénes: Operációs rendszerek
jegyzet/diasor és URL linkek alapján készült el

Témakörök

- UNIX processz - ismételés – *user ismeretek*.
- UNIX processz – *kernel ismeretek*.
- Végrehajtási (futási) mód váltása.
- Unix processzek ütemezés.
- Mintapéldák

UNIX processz – ismételés (felhasználói ismeretek)

„1. Mi a processz?

Feladataink végrehajtásához *programokat* kell írunk.

- A program maga a *végrehajtható kód*.
- A *processz* egy végrehajtás alatt lévő program.

Definíció: A processz – egy végrehajtási példánya egy párhuzamosságot nem tartalmazó végrehajtható programnak.

UNIX processz – ismételés (felhasználói ismeretek)

2. Hogyan indulnak a *processzek*?

- Rendszerinduláskor (rendszerfolyamatok, szolgáltatások).
- Felhasználó által.

UNIX processz – ismételés (felhasználói ismeretek)

3. Hogyan kezeljük a processzeket?

- *Listázás* (processz neve, azonosítója, futtatója, erőforrás adatok, ...)
 - `ps -a`, `ps axu`, `ps -u <felhasználó>`, ...
 - `top` és grafikus társai.
- *Vezérlés*
 - jelzésekkel: `kill <SIGNAL> <PID>`
 - leállítás, felfüggesztés.

Unix processzek adatai

a.) *PID* (Process ID): egyedi, a processz azonosító szám.

- PPID: szülő processz azonosítója.

b.) *A processz állapota*

- fut, alszik, stb.,
- ütemezési információk (prioritás, CPU használat, nice érték).

c.) *Hitelesítők*

- UID, GID: a kapcsolódó felhasználó adatai,
- valós és effektív azonosítók.

Unix processzek adatai

d.) *Memória-kezelési adatok*

- címleképezési térkép.

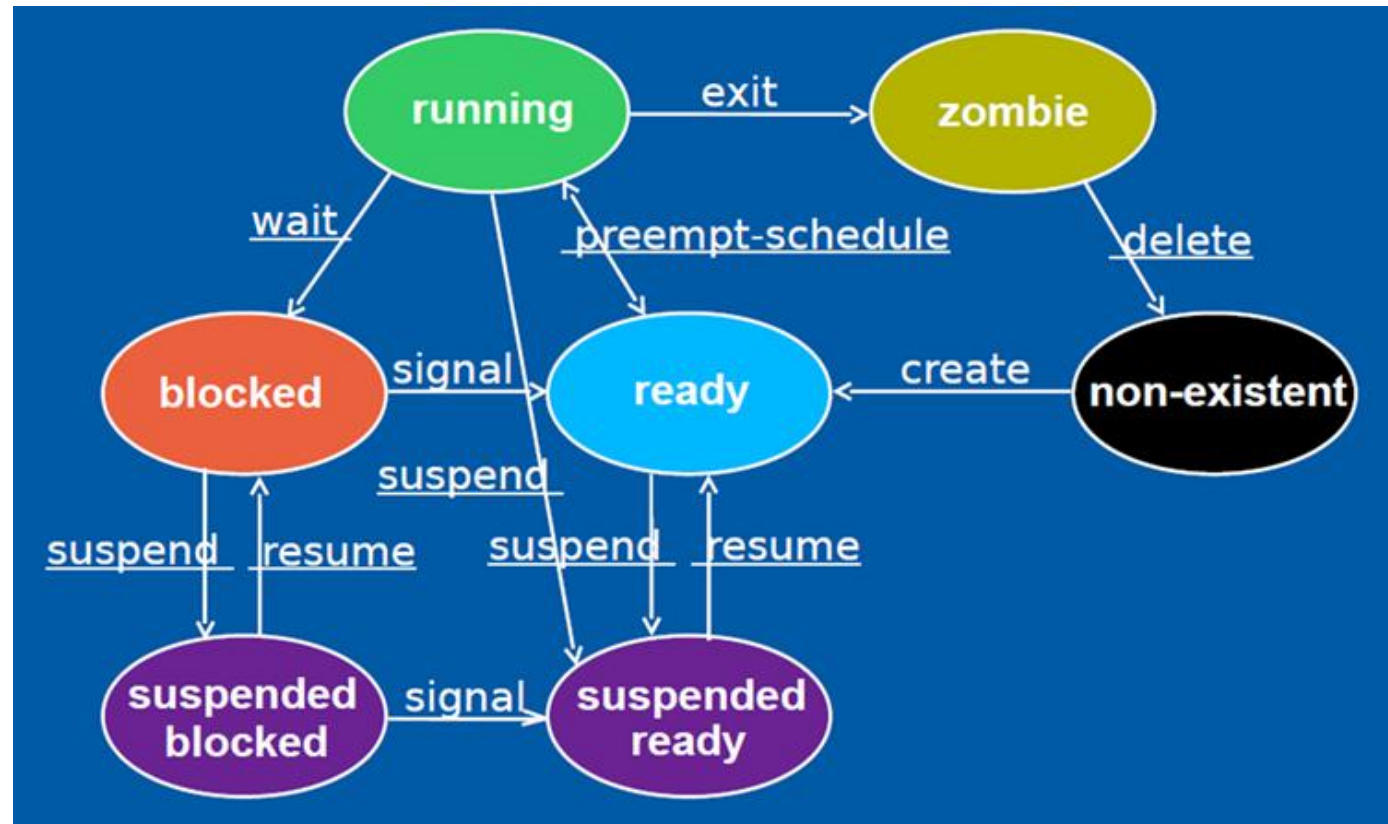
e.) *Kommunikációs adatok*

- fájlleírók, jelzés információk.

f.) *Statisztikák*

- erőforrás használat (számlázáshoz).

UNIX processzek állapotai – ismétlés



Unix processzek ütemezése

Processzek ütemezése – jellemzők

Ütemezés feladat: a futásra kész folyamatok közül a következő futót.

Ütemezés célja: a rendszer teljesítményét, a hardver kihasználtságát meghatározása.

UNIX processz ütemezés

Két csoportja:

- Klasszikus UNIX ütemező (tradicionális) – mi ezzel foglalkozunk.
- Modern UNIX ütemező.

UNIX ütemezése – jellemzői

Unix ütemezők: preemptív, prioritásos és időosztásos.

Preemptív: az OS elveheti a futás jogát egy processztől, *futásra kész állapotba* teszi a futó processzt és egy másik (futásra kész) processzt indít el.

Prioritásos:

- minden processz kap *egy prioritás értéket*,
- *a legnagyobb prioritású processzt* választja a futásra kész sorból.

UNIX ütemezése – jellemzői

Időosztásos:

- minden processz *kap egy időszeletet*, amíg futhat,
- az idő letelte után az *azonos prioritásúak közül* a következő fog futni.

UNIX ütemezése – jellemzői

Unix ütemezés esetén *a felhasználói, ill. kernel módban eltérő algoritmus:*

- *felhasználói módban:* preemptív és időosztásos, változó prioritás, körforgásos FCFS,
- *kernel módban:* nem preemptív, nincs időosztás, fix prioritású processzek.

UNIX processzek ütemezése

Processzek adatai:

- PID, PPID (folyamat- és szülőfolyamat azonosító),
- UID, GID,
- Aktuális állapot: f, FK, A, ... ,
- Ütemezési információk:
 - prioritás,
 - nice érték.

UNIX processzek prioritásának tárolása

A Unix processzek prioritásának tárolására *adatszerkezetek* használ.

- A *prioritás egész szám*: 0-127 közötti értéket vehet fel:
 - a 0 a *legmagasabb prioritás*, a 127 a *legkisebb*,
 - 0-49: *kernel prioritási szintek*,
 - 50-127: *felhasználói szintek*
- Az ütemező algoritmus a *processzeket prioritásuk szerint 32 bit FIFO sorba* rendezi.

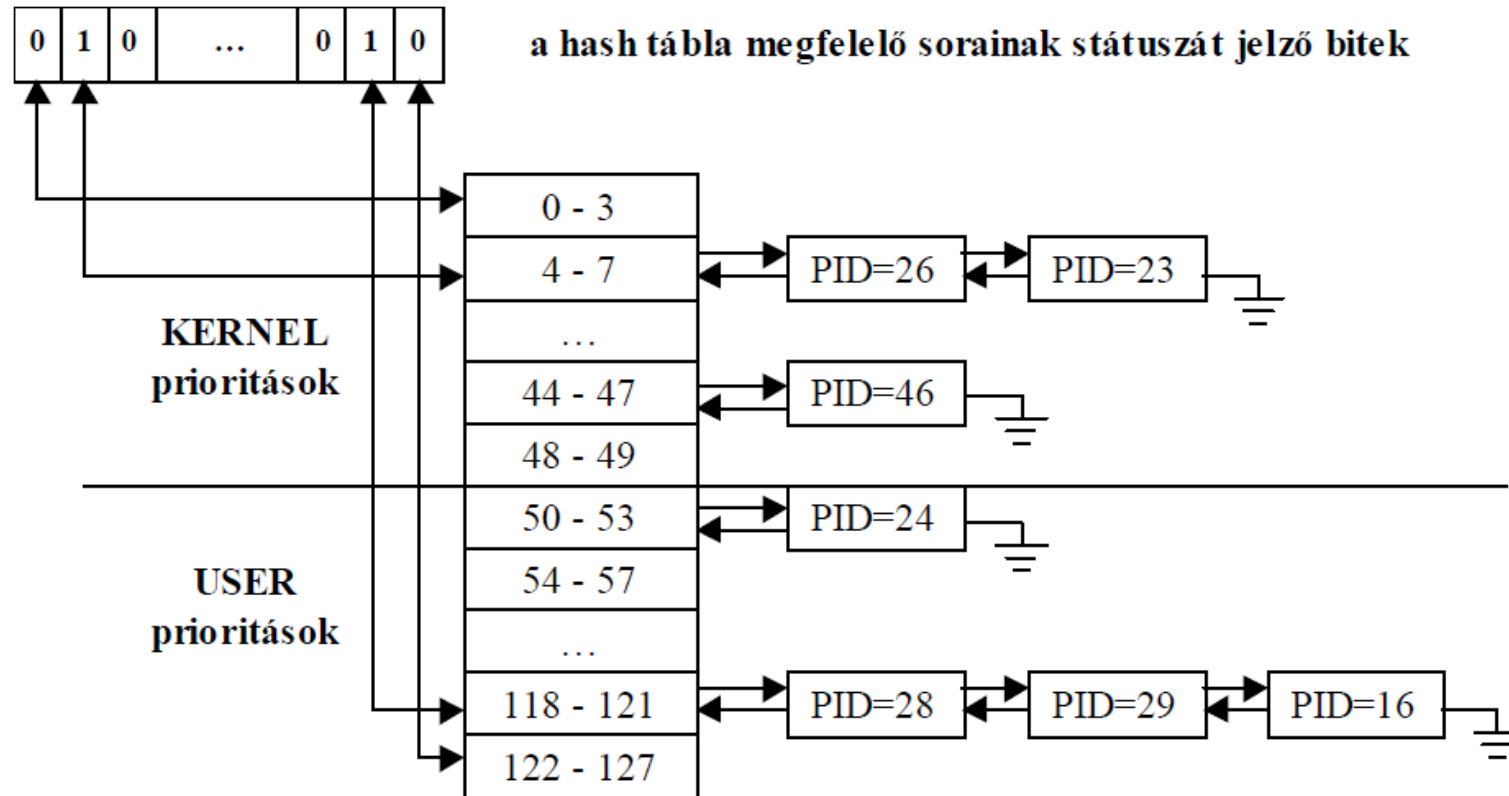
UNIX pocesszek ütemezési prioritása

0	legnagyobb prioritás	} KERNEL prioritások
.		
.		
.		
49		
50		
.		
.		
.		
127	legkisebb prioritás	} FELHASZNÁLÓI prioritások

Forrás: Benyó

UNIX process priority storage

The scheduler sorts processes by priority: **32 bit FIFO** queue.



Forrás: HorvathZS

Kernel módban ütemezés

Kernel módú ütemezés jellemzői:

- ütemezés *nem preemptív*,
- a prioritás *fix értékű*,
- a processzt nem lehet kényszeríteni, hogy lemondjon a CPU használatáról, pl. a nagyobb prioritású processz javára.
- *modern UNIX-ok: részben vagy teljesen preemptív.*

Kernel módú prioritás - alvási prioritás

Kernel módú prioritást a *processz elalvásának* az oka határozza meg. Ez az ún. *alvási prioritás* (sleep prio).

- A *processz felébredése* után ez fogja meghatározni a prioritását.
- Az *alvási prioritás* az *alvás okához* kötött, pl.:
 - 20 - diszk I/O-ra vár,
 - 28 - input-ra vár a karakteres terminálról.

User módban ütemezés – prioritás számítása

User módban az ütemezés preemptív.

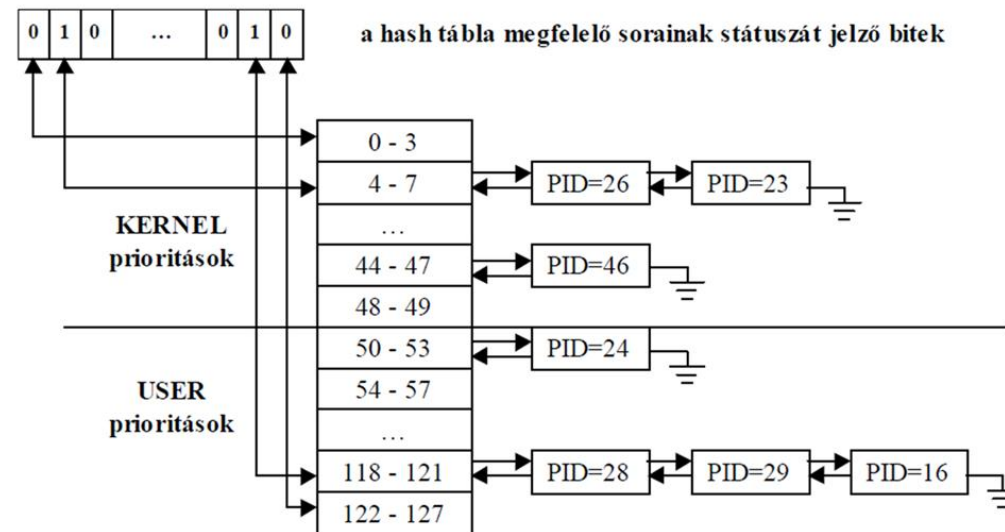
Mit jelent ez?: az OS elveheti a futás jogát egy processztől, futásra kész állapotba teszi.

Ütemezés alapja *felhasználói módban* - a *számított prioritás*:

- mindig a *legnagyobb prioritású processzt* ütemezi be a kernel,
- kiválasztja a *legmagasabb futási szintet*, ahol van processz.

User módban ütemezés – prioritás számítása

- ha *magasabb, mint a futó processz szintje*, akkor *újraütemez*.
- ha *azonos a futó processz szintjével*, és letelt a futó folyamat *időszelete*, akkor szintén *átütemez*: Round-Robin-os FCFS.



Round-Robin – ismételés

Round-Robin (RR, Körben járó, Körforgó)

- *Preemptív, időosztásos rendszerek alapja.*
- *Időszeleteket* (quantum) rendelünk a processzekhez.
- A kész állapotú processzek egy *soron nyilvántartottak*.
- Minden processz, amikor futni kezd kap egy *időszeletet*.
- Egy processz futhat, amíg a időszelete tart (ha blokkolódik, akkor addig sem fut).

Round-Robin – ismétlés

- Lejárt *szeletű processz a sor végére* kerül.
- A sor elején álló kapja a CPU-t. A listán a processzek “körben” járnak.

Előnye: egyszerű, kis költségű, garantáltan szóhoz jut minden processz.

Hátránya: nem vesz figyelembe fontosságot (!). Jelentős adminisztrációt igényel.

Round-Robin – ismétlés

- *Érdekesség: a válaszidő csökkenthető, ha kisebb időszeletet használ az operációs rendszer, de!*
- *Kisebb időszelet* → több CS → romló teljesítmény.
- *Nagy időszelet*: FCFS algoritmushoz hasonló lesz.

User módban - ütemezés adatai

A prioritás számításához *négy paraméter*t használ a UNIX:

- `p_pri` - a processz *aktuális prioritása*,
- `p_usrpri` - a processz user módban érvényes prioritása,
- `p_cpu` - a korábbi *CPU használatra vonatkozó szám*,
- `p_nice` - a user által adott prioritás módosító érték.

User módban - ütemezés adatai

- *User módból-Kernel módba váltáskor* a prioritás értéke „elmentődik” a `p_usrpri` változóba azért, hogy *user módba* visszalépéskor visszaállítható legyen.
- A `p_cpu` érték azt fejezi ki, hogy a processz az elmúlt *időszakban mennyi processzor időt* kapott. Ez csökkenteni fogja a prioritását.
- A `nice` szám (kedvezési szám): a felhasználó beavatkozása a prioritás számításába.

Prioritás számítása – user módban

- *Minden óraciklusban* a futó processzre növeli a `p_cpu` értékét

`p_cpu++`

- Minden *10. óraütésnél*
 - Round-Robin ütemezés a legmagasabb (user) futási szinten levő processzek között.

Prioritás számítása user módban

- *Minden 100. óraciklusban* a kernel kiszámítja a prioritás értékét:

- minden processznél „öregíti” a `p_cpu` értéket:

$$p_cpu = p_cpu * KF \text{ - ahol egy KF (korrekciós faktor)}$$

- kiszámolja a prioritást:

$$p_pri = P_USER + p_cpu/4 + 2 * p_nice$$

- a `P_USER` konstans érték, a kernel és a felhasználói módot választja szét:

$$P_USER = 50$$

Prioritás számítása user módban

P_USER *konstans*:

- a legmagasabb *user* módú prioritás: 50.

Korrektációs faktor (KF):

KF = várakozó folyamatok száma/várakozó folyamatok száma+1.

- Számos rendszerben *konstans az értéke.*

Pl.: 1, $\frac{1}{2}$, $\frac{2}{3}$, $\frac{3}{4}$...

User módú prioritás - összefoglalás

- *Minden óraütésnél*
 - Ha van magasabb prioritási sorban processz, akkor *átütemezés*.
- *Minden 10. óraütésnél*
 - Round-Robin ütemezés a legmagasabb (user) futási szinten levő processz között.

User módú prioritás - összefoglalás

- *Minden 100. óraütésnél*
 - a korrekciós faktor kiszámítása az elmúlt 100 ciklus átlagos terhelése alapján,
 - a p_cpu „öregítése”,
 - a prioritás újraszámolása,
 - szükség szerint átütemezés.

UNIX ütemezés - összefoglalás

Klasszikus UNIX ütemező (tradicionális):

- *felhasználói módban*: prioritásos, időosztásos, preemptív:
 - többszintű ütemezési tevékenység,
 - azonos prioritási szinten *round-robin* ütemezés (időosztás),
 - a prioritás alapja a *korábbi CPU használat* és a *nice* érték.

UNIX ütemezés - összefoglalás

- *kernel módban:*
 - rögzített prioritás, alapja az ún. *alvási prioritás*,
 - nem preemptív.
- egyszerű, kerüli az éhezést, de rosszul skálázható és nem támogat sok mai alkalmazási igényt (valós idejű rendszerek, multimédia, stb.)”[1],[2],[3],[3],[5]

Irodalom

1. Vadász Dénes: Operációs rendszerek, ME, 2007.

<http://www.iit.uni-miskolc.hu/~vadasz/GEIAL302B/2007-Ea>

2. Kóczy, Kondorossi szerk: Operációs rendszerek mérnöki megközelítésben, Panem, 2000.

3. Pannon Egyetem: Operációs rendszerek- ütemezés

https://dcs.uni-pannon.hu/files/docs/users/csertangyorgy/os/OS_05_Utemezes.pdf.

4. Micskei Zoltán: Operációs rendszerek, BME.

5. Mészáros Tamás: Unix folyamatok ütemezése, BME